

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

РЕФЕРАТ

по теме: «Управление и развертывание контейнеров в ОС Linux с использованием Docker и Kubernetes»

Выполнила:

студентка факультета Прикладной информатики

Гаджиева Патина Гасановна

Группа №К3420

Проверил:

доцент факультета Прикладной информатики

Ананченко Игорь Викторович

Санкт-Петербург

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ОСНОВЫ КОНТЕЙНЕРИЗАЦИИ И АРХИТЕКТУРА DOCKER.....	5
1.1 Концепция контейнеризации в Linux.....	5
1.2 Архитектура и компоненты Docker.....	6
1.3 Сетевые возможности и хранение данных в Docker	7
1.4 Docker Compose для управления приложениями.....	8
2 АРХИТЕКТУРА И КОМПОНЕНТЫ KUBERNETES	10
2.1 Общая архитектура кластера Kubernetes	10
2.2 Рабочие узлы и компоненты подов	11
2.3 Сервисы, развертывания и управление приложениями	12
2.4 Масштабирование и управление ресурсами	13
3 ОРКЕСТРАЦИЯ КОНТЕЙНЕРОВ.....	14
3.1 Принципы оркестрации контейнеров	14
3.2 Docker Swarm: архитектура и особенности.....	15
3.3 Сравнительный анализ Kubernetes и Docker Swarm.....	15
ЗАКЛЮЧЕНИЕ	17
СПИСОК ЛИТЕРАТУРЫ.....	19

ВВЕДЕНИЕ

Современные информационные системы характеризуются высокой динамичностью, масштабируемостью и необходимостью быстрого развертывания приложений в различных средах. Традиционные подходы к виртуализации, основанные на использовании полноценных виртуальных машин, уже не всегда удовлетворяют требованиям гибкости и эффективности, поскольку требуют значительных вычислительных ресурсов и времени на запуск. В этих условиях всё более широкое распространение получают технологии контейнеризации, позволяющие изолировать приложения на уровне операционной системы и обеспечивать их переносимость между различными платформами.

Контейнеры в Linux стали ключевым инструментом для разработчиков и системных администраторов благодаря своей лёгкости, скорости и удобству управления. Они позволяют запускать приложения в стандартизированной среде, исключая проблемы несовместимости библиотек и зависимостей. Наиболее популярным инструментом контейнеризации является Docker, предоставляющий удобные средства создания, распространения и управления контейнерами. Docker стал фактическим стандартом в индустрии, обеспечив единый формат контейнеров и простой интерфейс для работы с ними.

Однако по мере роста масштабов инфраструктуры и увеличения количества контейнеров возникает необходимость в автоматизации их развертывания, балансировки нагрузки, обновления и восстановления. Для решения этих задач используется Kubernetes — мощная система оркестрации контейнеров, разработанная Google. Kubernetes обеспечивает управление кластерами контейнеров, автоматическое масштабирование, мониторинг состояния приложений и распределение нагрузки, что делает его незаменимым инструментом в современных DevOps-процессах.

Актуальность темы обусловлена тем, что Docker и Kubernetes стали основой современной облачной инфраструктуры, микросервисной архитектуры и CI/CD-конвейеров. Их использование позволяет значительно ускорить разработку, тестирование и внедрение программных продуктов, обеспечивая при этом высокую надёжность и отказоустойчивость систем.

Целью данной работы является комплексное изучение технологий управления и развертывания контейнеров в ОС Linux с использованием Docker и Kubernetes. Работа охватывает теоретические основы контейнеризации, архитектурные компоненты обеих систем и практические аспекты их использования.

1 ОСНОВЫ КОНТЕЙНЕРИЗАЦИИ И АРХИТЕКТУРА DOCKER

1.1 Концепция контейнеризации в Linux

Контейнеризация в Linux — это подход к виртуализации, при котором приложения запускаются в отдельных изолированных средах, но при этом продолжают использовать одно общее ядро операционной системы. В отличие от виртуальных машин, где каждая копия системы включает собственную ОС и гипервизор, контейнеры работают значительно легче: они не требуют загрузки полноценной гостевой системы и используют встроенные в Linux механизмы изоляции. Благодаря этому на одном сервере можно разместить гораздо больше контейнеров, чем виртуальных машин при тех же ресурсах.

Основой контейнеризации служат два механизма ядра Linux — namespaces и cgroups. Namespaces отвечают за изоляцию различных аспектов среды: процессов, сетевых интерфейсов, точек монтирования, IPC-взаимодействия, пользователей и имени хоста. Процесс внутри контейнера видит только те ресурсы, которые ему «разрешены», и не может взаимодействовать с объектами других контейнеров или хоста. Cgroups, в свою очередь, позволяют контролировать потребление ресурсов: ограничивать использование процессора, памяти, диска и сети. Это защищает систему от ситуаций, когда один контейнер может перегрузить сервер и нарушить работу остальных приложений.

Если сравнивать контейнеры с виртуальными машинами, то преимущества контейнеризации становятся очевидными: контейнеры запускаются почти мгновенно, занимают меньше места на диске и требуют меньше оперативной памяти. Образы контейнеров легко распространять и версионировать. Однако у контейнеров есть и ограничения — они обеспечивают более слабую изоляцию, поскольку используют общее ядро. Поэтому виртуальные машины остаются актуальными там, где требуется строгая изоляция или запуск разных операционных систем на одном хосте. Контейнеры же оптимальны для

микросервисов и однородных приложений, работающих в рамках одной платформы.

1.2 Архитектура и компоненты Docker

Docker стал одной из самых популярных платформ для работы с контейнерами благодаря своей простоте и удобству. Его архитектура построена по принципу клиент–сервер. На стороне сервера работает Docker Engine — демон `dockerd`, который отвечает за создание, запуск и остановку контейнеров, управление образами, сетями и томами. Он принимает запросы от клиента и выполняет необходимые операции. Связь между клиентом и демоном осуществляется через REST API, что позволяет управлять Docker не только через командную строку, но и из внешних программ и автоматизированных систем.

Docker CLI — это инструмент командной строки, который предоставляет удобный интерфейс для взаимодействия с Docker Engine. Пользователь вводит команды, такие как `docker run`, `docker build` или `docker ps`, а CLI передаёт их демону, который выполняет соответствующие действия.

Центральным элементом Docker является образ. Образ представляет собой шаблон файловой системы, содержащий всё необходимое для запуска приложения: базовый слой операционной системы, библиотеки, зависимости, исполняемые файлы и конфигурацию. Образы состоят из нескольких слоёв, что делает их компактными и удобными для повторного использования. Например, если несколько образов используют один и тот же базовый слой, он хранится на диске только один раз. Это значительно экономит место и ускоряет сборку новых образов.

Контейнер — это запущенный экземпляр образа. При запуске к образу добавляется отдельный слой, в который записываются изменения, происходящие во время работы приложения. Этот слой является временным: если контейнер удалить, все изменения исчезнут. Именно поэтому контейнеры

считаются эфемерными, а образы — постоянной основой для их воспроизведения.

Управление контейнерами осуществляется через набор команд CLI. Команда `docker run` создаёт и запускает контейнер, позволяя указать порты, тома, переменные окружения и другие параметры. Команды `docker start`, `docker stop` и `docker restart` управляют жизненным циклом контейнеров. Команда `docker exec` позволяет выполнять команды внутри работающего контейнера, что удобно для диагностики и администрирования. Команда `docker logs` выводит журналы приложения, а `docker rm` и `docker rmi` удаляют контейнеры и образы, поддерживая чистоту окружения.

1.3 Сетевые возможности и хранение данных в Docker

Docker предоставляет гибкую систему сетевых драйверов, которые позволяют организовать взаимодействие контейнеров между собой и с внешним миром. Наиболее распространённым является драйвер `bridge`. Он создаёт виртуальный сетевой мост, к которому подключаются контейнеры. Каждый контейнер получает собственный IP-адрес, а доступ в интернет осуществляется через NAT. Такой подход обеспечивает изоляцию контейнеров и позволяет им взаимодействовать друг с другом в пределах одной виртуальной сети.

Драйвер `host` убирает сетевую изоляцию и позволяет контейнеру использовать сетевой стек хоста. Это повышает производительность, но снижает безопасность. Overlay-сети применяются в распределённых системах, например в Docker Swarm, и позволяют объединять контейнеры, работающие на разных узлах. Драйвер `macvlan` позволяет контейнерам получать собственные MAC-адреса и выглядеть как полноценные устройства в физической сети. Драйвер `none` полностью отключает сетевое взаимодействие контейнера, оставляя только `loopback`-интерфейс.

По умолчанию данные внутри контейнера хранятся в его файловой системе и исчезают при удалении контейнера. Чтобы обеспечить долговременное хранение, Docker использует тома. Тома управляются самим Docker и хранятся отдельно от контейнеров. Один том можно подключить к нескольким контейнерам, что удобно для общих данных. Альтернативой являются `bind mounts`, когда в контейнер монтируется директория хоста. Это удобно при разработке, но менее безопасно. Также поддерживаются `tmpfs`-монтирования, которые хранят данные в оперативной памяти.

Работа с томами осуществляется через команды `docker volume create` и параметры `-v` или `--mount` при запуске контейнера. В многоконтейнерных приложениях тома и сети часто описываются в конфигурации Docker Compose.

1.4 Docker Compose для управления приложениями

Docker Compose — это инструмент, который позволяет описывать и запускать многоконтейнерные приложения с помощью файла `docker-compose.yml`. Вместо ручного запуска каждого контейнера с множеством параметров администратор описывает все сервисы, их зависимости, сети и тома в одном декларативном документе. Это особенно удобно для микросервисных архитектур, где приложение состоит из нескольких взаимосвязанных компонентов: веб-интерфейса, API, базы данных, кеша, брокера сообщений и т.д..

В конфигурации Docker Compose, как правило, присутствуют разделы `services`, `networks` и `volumes`. В секции `services` описывается каждый сервис: указывается образ (или путь к Dockerfile для сборки), порты, переменные окружения, подключаемые тома, сети и зависимости. Параметр `depends_on` задает порядок запуска зависимых сервисов, что позволяет, например, сначала поднять базу данных, а затем само приложение. Раздел `volumes` объявляет именованные тома, которые затем подключаются к контейнерам; это обеспечивает удобное и единообразное управление

персистентными данными. Секция `networks` позволяет создавать отдельные сети для изоляции разных частей системы.

Основными командами являются `docker-compose up` и `docker-compose down`. Первая команда создает и запускает все описанные сервисы, автоматически создавая необходимые сети и тома, а вторая — останавливает и удаляет контейнеры, сети и другие объекты (по желанию оставляя тома). Для масштабирования отдельного сервиса можно использовать параметр `--scale`, например: `docker-compose up --scale web=3`, что запустит три экземпляра веб-сервиса. В среде разработки нередко используются `bind mounts`, чтобы изменения в исходном коде автоматически применялись внутри контейнера без пересборки образа. В целом, Docker Compose существенно упрощает управление многоконтейнерными приложениями, особенно в `dev`- и `test`-окружениях.

2 АРХИТЕКТУРА И КОМПОНЕНТЫ KUBERNETES

2.1 Общая архитектура кластера Kubernetes

Kubernetes сегодня считается стандартом для управления контейнеризованными приложениями. Его архитектура построена таким образом, чтобы обеспечить надёжность, масштабируемость и удобство эксплуатации в условиях распределённых систем. В основе лежит разделение на управляющую плоскость (Control Plane) и рабочие узлы (Worker Nodes). Управляющая плоскость отвечает за принятие решений о состоянии всего кластера: она планирует, где будут размещены новые контейнеры, следит за количеством реплик, выполняет обновления и обеспечивает согласованность конфигурации. Рабочие узлы, напротив, выполняют прикладную часть — именно на них запускаются контейнеры с приложениями, которые обслуживают пользователей.

Центральным элементом управляющей плоскости является компонент `kube-apiserver`. Он выступает в роли «фронт-энда» для всех запросов к кластеру: именно через него проходят операции по созданию, изменению и удалению объектов — будь то поды, сервисы или конфигурации. API-сервер проверяет корректность запросов, применяет политики безопасности и взаимодействует с хранилищем `etcd`. `Etcd` представляет собой отказоустойчивое распределённое `key-value` хранилище, где сохраняется текущее состояние кластера. Благодаря этому любое изменение фиксируется и становится доступным всем компонентам системы.

Другим важным элементом является планировщик `kube-scheduler`. Его задача — отслеживать новые поды, которые ещё не назначены на конкретный узел, и выбирать подходящий узел для их запуска. При этом учитываются доступные ресурсы, ограничения и правила, заданные администратором. Контроллеры, которыми управляет `kube-controller-manager`, следят за тем, чтобы

реальное состояние кластера соответствовало описанному в конфигурации. Например, если указано, что должно быть три реплики пода, контроллер создаст недостающие экземпляры или удалит лишние.

2.2 Рабочие узлы и компоненты подов

Рабочий узел Kubernetes — это физический или виртуальный сервер, на котором запускаются контейнеры приложений. Каждый узел содержит несколько ключевых компонентов. Kubelet — агент, который взаимодействует с API-сервером, получает спецификации подов и следит за тем, чтобы на узле были запущены контейнеры в соответствии с этими спецификациями. Он также отслеживает состояние контейнеров и передаёт информацию обратно в управляющую плоскость. Kube-proxy отвечает за сетевую логику сервисов на уровне узла: он настраивает правила iptables или IPVS, чтобы маршрутизировать трафик к нужным подам. Наконец, контейнерный runtime (например, containerd или CRI-O) выполняет фактический запуск контейнеров на основе образов.

Минимальной единицей развертывания в Kubernetes является pod. Под можно рассматривать как логический контейнер для одного или нескольких тесно связанных контейнеров. Контейнеры внутри одного пода разделяют сетевое пространство — у них общий IP-адрес и порты, а также могут совместно использовать тома для хранения данных. Обычно под содержит один основной контейнер приложения, но часто применяется шаблон sidecar, когда дополнительный контейнер выполняет вспомогательные функции, например логирование или проксирование. Поды считаются временными сущностями: при сбоях или обновлениях они могут быть уничтожены и созданы заново. Поэтому хранить важное состояние внутри пода не рекомендуется — для этого используются внешние тома или специализированные сервисы хранения.

Каждому поду выделяется уникальный IP-адрес в рамках кластера, что упрощает внутреннюю коммуникацию между сервисами. Однако этот адрес не

является постоянным и меняется при пересоздании пода. Чтобы обеспечить стабильную точку доступа, Kubernetes использует более высокоуровневые абстракции — Service и Ingress.

2.3 Сервисы, развертывания и управление приложениями

Сервис в Kubernetes — это объект, который обеспечивает стабильную точку доступа к динамическому набору подов. Он имеет постоянный виртуальный IP-адрес и DNS-имя, что позволяет другим компонентам системы обращаться к нему независимо от того, какие именно поды находятся за ним в данный момент. Сервис определяет селектор меток, по которому автоматически находит поды, входящие в его пул, и распределяет между ними трафик. Существует несколько типов сервисов: ClusterIP для внутреннего доступа, NodePort для открытия порта на каждом узле и доступа извне, LoadBalancer для интеграции с внешним балансировщиком облачного провайдера и ExternalName для перенаправления к внешнему DNS-имени.

Для управления реплицируемыми приложениями используются развертывания (Deployments). В манифесте Deployment описывается желаемое количество реплик подов, шаблон пода и стратегия обновления. Контроллер Deployment следит за тем, чтобы фактическое количество подов соответствовало указанному. При обновлении версии образа Kubernetes выполняет rolling update: постепенно создаёт новые поды и удаляет старые, сохраняя доступность приложения. В случае проблем возможен быстрый откат к предыдущей версии.

Ingress — это объект, который описывает правила маршрутизации HTTP(S)-трафика извне кластера к внутренним сервисам. В отличие от LoadBalancer, который обычно предоставляет один внешний IP на один сервис, Ingress позволяет настраивать маршруты для множества сервисов, исходя из доменного имени и пути в URL. Для работы Ingress необходим Ingress Controller — компонент, который анализирует правила и конфигурирует балансировщик

нагрузки. Ingress также поддерживает SSL/TLS-терминацию, что упрощает управление безопасным доступом.

2.4 Масштабирование и управление ресурсами

Одним из важнейших преимуществ Kubernetes является возможность автоматического масштабирования приложений. Horizontal Pod Autoscaler отслеживает использование ресурсов и автоматически изменяет количество реплик подов, чтобы поддерживать заданный уровень загрузки. Например, если средняя загрузка процессора превышает порог, НРА увеличивает число подов, а при снижении нагрузки — уменьшает. Это позволяет приложениям адаптироваться к изменению трафика и эффективно использовать ресурсы.

Kubernetes также предоставляет развитые средства управления ресурсами. Для каждого контейнера можно указать requests и limits: первое определяет гарантированный минимум ресурсов, второе — верхнюю границу. При превышении лимита по памяти контейнер может быть завершён системой, а при ограничении CPU ему будет выделяться только часть процессорного времени. На уровне namespaces используются объекты ResourceQuota и LimitRange, которые позволяют ограничивать суммарное потребление ресурсов и задавать типовые значения.

Namespaces служат механизмом логической изоляции ресурсов внутри кластера. Они позволяют делить единый кластер между несколькими командами или проектами, не создавая отдельные кластеры для каждого случая. Взаимодействие с ресурсами в определённом namespace осуществляется через утилиту kubectl с явным указанием пространства имён. Совместное использование namespaces с механизмами управления ресурсами и рольно-базируемым контролем доступа позволяет строить безопасные и многопользовательские кластеры.

3 ОРКЕСТРАЦИЯ КОНТЕЙНЕРОВ

3.1 Принципы оркестрации контейнеров

С развитием микросервисных архитектур количество контейнеров, необходимых для работы одного приложения, может стремительно расти. В крупных системах число контейнеров нередко достигает сотен или даже тысяч, и их ручное сопровождение становится практически невозможным. В таких условиях возникает необходимость в специализированных инструментах, которые способны автоматически управлять жизненным циклом контейнеров. Именно эту задачу решает оркестрация контейнеров.

Под оркестрацией понимают комплекс механизмов, обеспечивающих автоматическое развертывание, масштабирование, обновление и мониторинг контейнеризованных приложений. Оркестратор распределяет контейнеры по узлам кластера, следит за их состоянием, обеспечивает балансировку нагрузки и автоматически восстанавливает сервисы в случае сбоев. Таким образом, оркестрация позволяет поддерживать стабильность и предсказуемость работы распределённых систем без постоянного вмешательства администратора.

Ключевые задачи оркестрации включают управление конфигурацией приложений, поддержание заданного числа экземпляров сервисов, обеспечение отказоустойчивости, автоматическое масштабирование и предоставление унифицированных механизмов сетевого взаимодействия. Современные оркестраторы, такие как Kubernetes и Docker Swarm, используют декларативный подход: администратор описывает желаемое состояние системы, а оркестратор самостоятельно приводит реальное состояние в соответствие с этим описанием. Такой подход значительно снижает сложность сопровождения сложных распределённых систем и делает инфраструктуру более предсказуемой.

3.2 Docker Swarm: архитектура и особенности

Docker Swarm представляет собой встроенный в Docker механизм оркестрации, который позволяет объединять несколько Docker-хостов в единый кластер и управлять контейнерными сервисами на нём. Архитектура Swarm основана на разделении узлов на управляющие (manager) и рабочие (worker). Управляющие узлы отвечают за поддержание состояния кластера, распределение задач и обеспечение консенсуса с использованием протокола Raft. Рабочие узлы выполняют контейнеры и предоставляют вычислительные ресурсы.

Сервисы в Docker Swarm описываются аналогично обычным контейнерам Docker, но с возможностью указания числа реплик, стратегий обновления и других параметров. Это позволяет легко масштабировать приложения и управлять их обновлениями. Одним из преимуществ Swarm является его тесная интеграция с Docker CLI: пользователи, знакомые с Docker, могут быстро освоить управление кластером. Для инициализации кластера достаточно выполнить команду `docker swarm init`, после чего новые узлы могут быть добавлены с помощью автоматически сгенерированного токена.

Swarm предоставляет встроенные механизмы балансировки нагрузки и сервис-дискавери, основанные на DNS-именах. Однако, несмотря на удобство и простоту, возможности Swarm в области масштабирования, управления ресурсами и расширяемости заметно уступают Kubernetes. Он подходит для небольших и средних проектов, но при росте инфраструктуры его функциональность может оказаться недостаточной.

3.3 Сравнительный анализ Kubernetes и Docker Swarm

Сравнение Kubernetes и Docker Swarm целесообразно проводить по нескольким ключевым параметрам: сложности развертывания,

масштабируемости, гибкости управления и доступности экосистемы инструментов.

Docker Swarm выигрывает по простоте: кластер можно развернуть буквально за несколько минут, используя всего несколько команд. Он органично вписывается в привычный рабочий процесс Docker и не требует глубокого изучения новых концепций. Это делает Swarm привлекательным для небольших команд и проектов, где важна скорость и минимальная сложность.

Kubernetes, напротив, обладает более сложной архитектурой и требует большего объёма знаний для настройки и эксплуатации. Однако эта сложность оправдывается его возможностями. Kubernetes предоставляет развитые механизмы автоматического масштабирования, гибкую систему управления ресурсами, поддержку пространств имён для изоляции окружений и мощные средства обеспечения безопасности. Он способен эффективно работать в кластерах, состоящих из сотен узлов, и поддерживает сложные сценарии развертывания.

Экосистема Kubernetes значительно шире: существует множество инструментов для мониторинга, логирования, CI/CD, сервис-мешей и сетевых политик, ориентированных именно на Kubernetes. Благодаря этому Kubernetes стал стандартом для крупных корпоративных систем, где важны масштабируемость, отказоустойчивость и гибкость.

В итоге Docker Swarm остаётся удобным решением для небольших и средних проектов, где требуется простота и минимальная конфигурация. Kubernetes же является предпочтительным выбором для крупных распределённых систем и enterprise-инфраструктур, предъявляющих высокие требования к надёжности и управляемости.

ЗАКЛЮЧЕНИЕ

Технологии контейнеризации и оркестрации, представленные Docker и Kubernetes, существенно преобразовали подходы к созданию, тестированию и эксплуатации программных систем. Docker сделал процесс упаковки приложений в единообразные образы значительно проще, обеспечив удобный механизм переноса и запуска программного обеспечения в среде Linux. Kubernetes дополнил эти возможности, предоставив мощную платформу для управления контейнерными рабочими нагрузками, автоматизируя развертывание, масштабирование и обновление приложений. Совместное использование этих инструментов стало основой современных практик DevOps и CI/CD, позволяя ускорить выпуск обновлений и повысить устойчивость сервисов.

Анализ архитектуры Docker показал, что его эффективность во многом определяется продуманной моделью образов и контейнеров, многослойной файловой системой и удобством взаимодействия через интерфейс командной строки. Docker Compose, в свою очередь, предоставляет разработчикам удобный способ описания и запуска многокомпонентных приложений, что особенно важно на этапах разработки и тестирования. Однако для крупных распределённых систем требуются более развитые механизмы управления, которые предоставляет Kubernetes. Его архитектура, включающая управляющую плоскость и рабочие узлы, а также использование таких объектов, как Pods, Services, Deployments и Ingress, обеспечивает высокий уровень гибкости, масштабируемости и надёжности при эксплуатации микросервисных приложений.

Сравнение Kubernetes и Docker Swarm показывает, что несмотря на простоту и удобство Swarm, именно Kubernetes стал основным инструментом оркестрации в индустрии. Это объясняется его широкими функциональными возможностями, развитой экосистемой и поддержкой со стороны ведущих

облачных платформ. Тем не менее выбор конкретного решения зависит от масштаба и требований проекта: для небольших систем, где важны простота и минимальная конфигурация, Swarm или базовый Docker могут быть вполне достаточными.

Таким образом, управление и развертывание контейнеров в Linux с использованием Docker и Kubernetes представляет собой важное направление для специалистов, работающих в области разработки и эксплуатации программного обеспечения. Владение этими технологиями позволяет создавать гибкие, масштабируемые и устойчивые системы, способные эффективно работать в условиях высокой нагрузки и быстро меняющихся требований. Можно ожидать, что в дальнейшем экосистема контейнеризации будет продолжать развиваться, появятся новые инструменты и подходы, однако базовые принципы, рассмотренные в данной работе, сохранят своё значение как фундамент современной облачной и микросервисной архитектуры.

СПИСОК ЛИТЕРАТУРЫ

1. Введение в Docker: образы, контейнеры и докер-файлы // 1cloud.ru.
URL: https://1cloud.ru/blog/docker_start (дата обращения: 12.12.2025).
2. Как устроен Docker и почему он популярен // Yandex Cloud.
URL: <https://yandex.cloud/ru/blog/docker-guide> (дата обращения: 1.12.2025).
3. Оркестрация контейнеров промышленного уровня. Документация Kubernetes. URL: <https://kubernetes.io/ru/> (дата обращения: 12.12.2025).
4. Компоненты Kubernetes. Документация Kubernetes.
URL: <https://kubernetes.io/ru/docs/concepts/overview/components/> (дата обращения: 12.12.2025).
5. Что такое оркестрация контейнеров // Selectel.
URL: <https://selectel.ru/blog/orchestration/> (дата обращения: 12.12.2025).
6. Kubernetes vs Docker: в чем различия // Selectel.
URL: <https://selectel.ru/blog/kubernetes-vs-docker/> (дата обращения: 12.12.2025).
7. Управление ресурсами в Kubernetes // Блог Батищев А.
URL: <https://www.batishchev.ru/blog/all/upravlenie-resursami-v-k8s/> (дата обращения: 12.12.2025).
8. Обзор способов защиты контейнеров Docker // Selectel, Habr.
URL: <https://habr.com/ru/companies/selectel/articles/854850/> (дата обращения: 12.12.2025).
9. Как повысить безопасность Docker-контейнеров // Timeweb Cloud.
URL: <https://timeweb.cloud/tutorials/docker/kak-povysit-bezopasnost-docker> (дата обращения: 12.12.2025).
10. Kubernetes VS Docker Swarm – в чем разница? // Merionet Wiki.
URL: <https://wiki.merionet.ru/articles/kubernetes-vs-docker-swarm-v-chem-raznica> (дата обращения: 12.12.2025).